2 parcial

Universidad: Politécnica de Atlautla

Programa: Ing. en Tecnologías de la Información e Innovación Digital

Asignatura: Fundamentos de Inteligencia Artificial

Tema: Actividad 2.3 Búsqueda Local

Docente: Mtro. Javier García Lara

Alumno: Anahi Fuentes Avendaño

Enlace de GitHub:
https://github.com/fana60203/FIA_G171_Anahi_Fuentes

Grupo: 171

Fecha de entrega:

22 – Octubre - 2026

Actividad 2.3

Implementación + Optimización. Implementar Hill Climbing y Simulated Annealing para optimizar una función real (ej. Ruta mínima, función de costo). Reportar: enunciado, función objetivo, vecindad, parámetros (T, iteraciones, n), Grafica de convergencia y comparación de resultados.

Planteamiento del problema

Una empresa de paquetería necesita planear una gira de distribución que visite una sola vez cada una de **25 ciudades de México** y regrese a la ciudad de partida. El objetivo es encontrar el recorrido cuya longitud total sea la menor posible. Es una instancia del problema del viajante (TSP), que es NP-difícil: con 25 ciudades existen (25-1)! /2, es decir, más de 3×1023 recorridos distintos, por lo que revisar todos es inviable y se recurre a métodos de búsqueda local.
Datos de entrada. Coordenadas aproximadas (latitud, longitud) de 25 ciudades: Ciudad de México, Guadalajara, Monterrey, Puebla, Querétaro, Mérida, Tijuana, Chihuahua, Oaxaca, Veracruz, León, Hermosillo, Cancún, Villahermosa, Tuxtla Gutiérrez, Acapulco, Morelia, San Luis Potosí, Zacatecas, Durango, Culiacán, La Paz, Saltillo, Tampico y Torreón. Las distancias se calculan con la fórmula de Haversine (distancia en línea recta sobre la esfera terrestre, radio 6371 km), por lo que son una aproximación de la distancia real por carretera.

Formulación

Elementos	**Definición**
Estado	Permutación de las 25 ciudades (orden de visita), circulo cerrado
Función objetivo	f(ruta) = suma de d(ruta[i+1]) para i = 0..24, con regreso a la primera ciudad (km). Se minimiza
Vecindario	2-opt: se elige un segmento de la ruta y se invierte. Cada ruta tiene 276 vecinos (sin contar el inicio fijo).
Estado inicial	Permutación aleatoria (costo de referencia: 24114 km con la semilla 0).
Restricciones	Cada ciudad exactamente una vez, la permutación garantiza factibilidad en todo momento.

Procedimientos

Hill Climbing (ascenso de colina, descenso en minimización). Desde la solución actual se evalúan todos los vecinos 2-opt y se pasa al mejor si mejora el costo; el algoritmo termina cuando ningún vecino mejora (óptimo local). Se

probaron dos variantes: *HC simple* (un arranque aleatorio) y *HC con 5 reinicios* aleatorios, conservando el mejor resultado.

Simulated Annealing (recocido simulado). En cada iteración se genera un vecino 2-opt aleatorio; si mejora, se acepta; si empeora en delta km, se acepta con probabilidad exp(-delta/T) (criterio de Metropolis). La temperatura se enfría de forma geométrica: T = alpha × T. Esto permite escapar de óptimos locales al inicio y comportarse como Hill Climbing al final.

Parámetros utilizados

Parámetro	Valor	Descripción
T0 (temperatura inicial)	2000	Del orden de los incrementos de costo típicos de un movimiento
Alpha (factor de enfriamiento, h)	0.9995	T se multiplica por Alpha en cada iteración
Iteraciones máximas	40000	Con Tmin = 0.001 como criterio adicional de
Reinicios de HC	5	Arranques aleatorios independientes
Semillas	30 (0 a 29)	Repeticiones independientes para obtener estadísticas

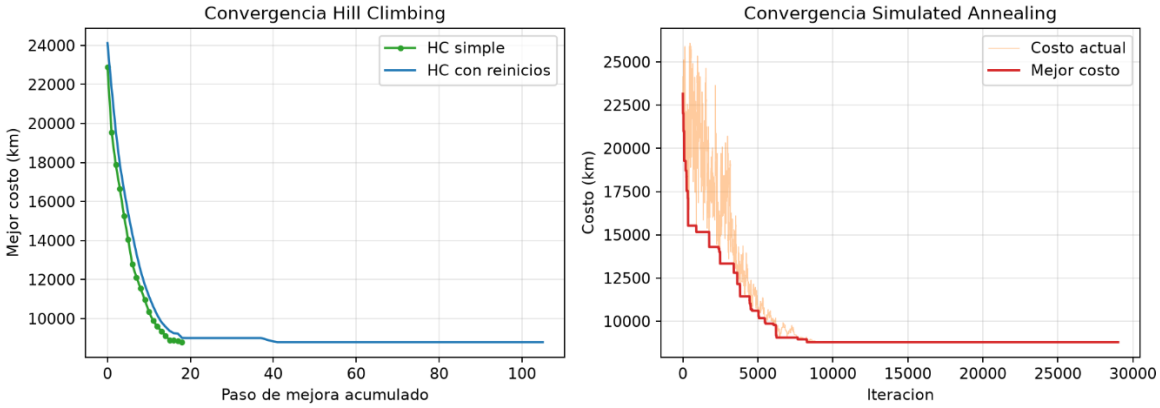
Resultados

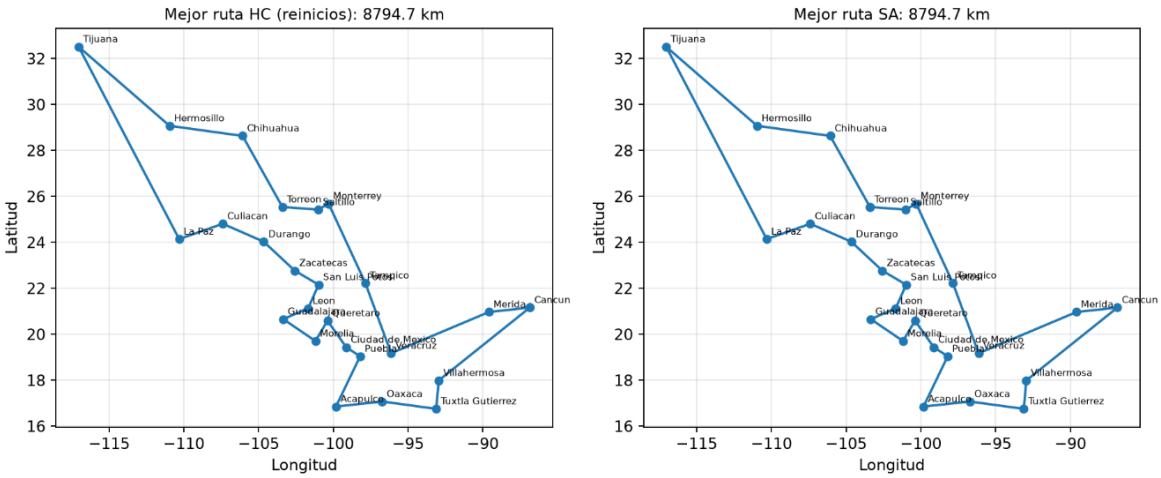
Cada algoritmo se ejecuto con 30 semillas distintas. La tabla resume el costo final (km) y el tiempo de cómputo promedio.

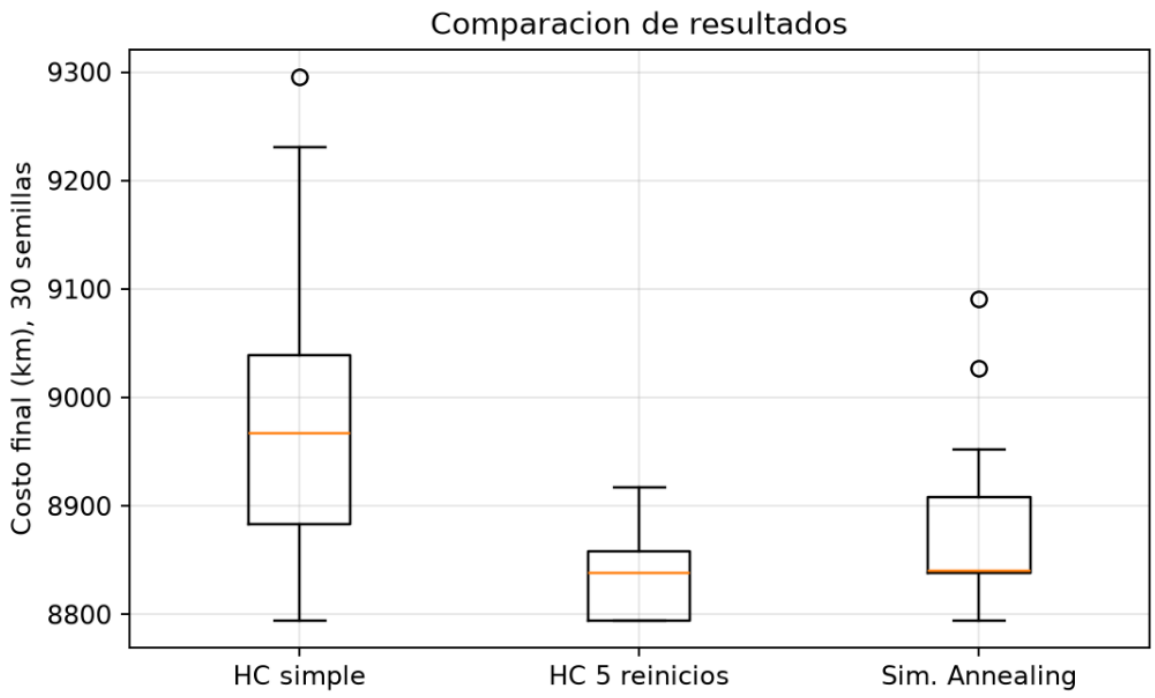
Algoritmo	Mejor (km)	Media (km)	Peor (km)	Desv.est.	Veces que hallo el mejor	Tiempo medio (S)
Hill Climbing simple	8794.7	8979.3	9295.7	125.4	1 de 30	0.014
Hill Climbing	8794.7	8835.2	8917.5	32.5	9 de 30	0.073

(5 reinicios)						
Simulated Annealing	8794.7	8871.9	9090.9	65.7	4 de 30	0.149

Mejor solución encontrada: **8794.7 km**, frente a 24114 km de una ruta aleatoria (reducción de 63.5%).
Mejor ruta (SA): Torreon -> Saltillo -> Monterrey -> Tampico -> Veracruz -> Merida -> Cancun -> Villahermosa -> Tuxtla Gutierrez -> Oaxaca -> Acapulco -> Puebla -> Ciudad de Mexico -> Queretaro -> Morelia -> Guadalajara -> Leon -> San Luis Potosi -> Zacatecas -> Durango -> Culiacan -> La Paz -> Tijuana -> Hermosillo -> Chihuahua -> (regreso a Torreon).







Sensibilidad de Simulated Annealing a T0 y aplha

Se probaron cuatro configuraciones (15 semillas, 40,000 iteraciones máximas)

T0	alpha	Costo medio (km)	Mejor costo(km)
50	0.99	9109.7	8840.3
500	0.99	9002.3	8794.7
2000	0.995	9023.3	8838.3
2000	0.999	8895.2	8794.7

Un enfriamiento más lento (alpha = 0.999) dio el mejor costo medio, a cambio de más iteraciones útiles; una temperatura inicial muy baja (T0 = 50) se comporta casi como Hill Climbing y obtuvo la peor media.

Análisis

- **Óptimos locales.** El Hill Climbing simple tardó solo unos 0.014 s y convergió en ~18 pasos, pero quedó atrapado en óptimos locales: su costo medio fue 8979 km y solo una de 30 corridas alcanzó el mejor valor (8794.7 km).

- **Reinicios**. Con 5 reinicios la media bajó a 8835 km y la desviación a 32.5 km, con el mejor valor en 9 de 30 corridas, a un costo de un 5 veces más tiempo.
- **Simulated Annealing**. Obtuvo una media de 8872 km (mejor que HC simple) y halló el mejor valor en 4 de 30 corridas. En la Figura 1 se ve cómo al inicio acepta soluciones peores (curva naranja muy ruidosa) y luego se estabiliza al bajar la temperatura. Con estos parámetros no superó al HC con reinicios en media; su ventaja crece cuando el espacio es mayor o el vecindario tiene muchos óptimos locales.
- **Complejidad.** Cada paso de HC evalúa O(n2) vecinos; SA evalúa un vecino por iteración, con costo O(n) por evaluación al recalcular la ruta completa (se podría reducir a O (1) calculando solo la diferencia de aristas).
- **Limitaciones.** Las distancias son en línea recta (Haversine) y no reflejan la red carretera. No se puede garantizar que 8794.7 km sea el óptimo global, aunque los tres métodos coinciden en esa mejor solución.

Conclusiones

En esta actividad busqué la ruta más corta para visitar 25 ciudades de México y volver al punto de inicio. Probé dos métodos: uno que siempre elige el cambio que mejora la ruta (Hill Climbing) y otro que a veces acepta cambios peores para no quedarse atorado (Simulated Annealing).

Con los dos logré bajar el recorrido de 24,114 km a 8,794.7 km, casi 64% menos. El primer método es muy rápido, pero depende de dónde empiece y muchas veces se queda con una solución buena, aunque no la mejor. Cuando lo repetí varias veces desde puntos distintos, los resultados mejoraron bastante. El segundo método tardó más, pero también llegó a la mejor ruta y mostró que aceptar algunos cambios peores al inicio ayuda a explorar más opciones.

Aprendí que no hay un método perfecto: uno es más rápido y el otro más flexible, y depende de lo que se necesite. También me quedó claro que estos métodos sirven mucho cuando revisar todas las rutas posibles sería imposible. Una limitación es que usé distancias en línea recta y no por carretera, así que en la vida real la ruta tendría que ajustarse.

Código Python

""" Actividad Parcial 2 - Seccion 2.3 Busqueda Local

Fundamentos de Inteligencia Artificial - Politecnica de Atlautla

Problema: Ruta minima (TSP) para una gira de distribucion que visita 25 ciudades de Mexico y regresa al punto de partida.

Algoritmos: Hill Climbing (con reinicios aleatorios) y Simulated Annealing.

"""

```
import math
 import random
import json
import statistics
import time
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
 # ----------------------------------------------------------------------
# 1. DATOS DEL PROBLEMA: ciudades (latitud, longitud) aproximadas
# ----------------------------------------------------------------------
CIUDADES = {
 "Ciudad de Mexico": (19.4326, -99.1332),
 "Guadalajara": (20.6597, -103.3496),
"Monterrey": (25.6866, -100.3161),
"Puebla": (19.0414, -98.2063),
"Queretaro": (20.5888, -100.3899),
"Merida": (20.9674, -89.5926)
, "Tijuana": (32.5149, -117.0382),
"Chihuahua": (28.6330, -106.0691),
 "Oaxaca": (17.0732, -96.7266),
 "Veracruz": (19.1738, -96.1342),
```

```
 "Leon": (21.1250, -101.6860),
 "Hermosillo": (29.0729, -110.9559),
"Cancun": (21.1619, -86.8515),
"Villahermosa": (17.9892, -92.9475),
 "Tuxtla Gutierrez": (16.7528, -93.1152),
"Acapulco": (16.8531, -99.8237),
 "Morelia": (19.7060, -101.1950),
 "San Luis Potosi": (22.1565, -100.9855),
"Zacatecas": (22.7709, -102.5832),
"Durango": (24.0277, -104.6532),
"Culiacan": (24.8091, -107.3940),
 "La Paz": (24.1426, -110.3128),
 "Saltillo": (25.4232, -101.0053),
 "Tampico": (22.2331, -97.8610),
"Torreon": (25.5428, -103.4068),
 }
#Lista de nombres (para indexar) y numero total de ciudades
NOMBRES = list(CIUDADES.keys())
N = len(NOMBRES)


def haversine(a, b):
"""Distancia en km entre dos coordenadas (lat, lon) sobre una esfera."""
R = 6371.0 # radio medio de la Tierra en km
# Convertimos grados a radianes (a = (lat1, lon1), b = (lat2, lon2))
la1, lo1, la2, lo2 = map(math.radians, (*a, *b))
# Termino "haversine" de la formula
d = math.sin((la2 - la1) / 2) ** 2 + \
math.cos(la1) * math.cos(la2) * math.sin((lo2 - lo1) / 2) ** 2
return 2 * R * math.asin(math.sqrt(d))


# Matriz de distancias precalculada: DIST[i][j] = km entre la ciudad i y la j.
# Se calcula una sola vez para que evaluar una ruta sea rapido.
```

```
DIST = [[haversine(CIUDADES[i], CIUDADES[j]) for j in NOMBRES] for i in
NOMBRES]


# ----------------------------------------------------------------------
# 2. FUNCION OBJETIVO Y VECINDARIO
# ----------------------------------------------------------------------
def costo(ruta):
"""Funcion objetivo: longitud total del circuito cerrado (km).
ruta es una lista con los indices de las ciudades en orden de visita."""
# (i + 1) % N hace que la ultima ciudad regrese a la primera (circuito cerrado)
return sum (DIST[ruta[i]][ruta[(i + 1) % N]] for i in range(N))


def vecino_2opt(ruta, i, j):
"""Vecino 2-opt: invierte el segmento ruta[i..j].
Siempre produce una permutacion valida (cada ciudad una sola vez)."""
return ruta[:i] + ruta [i:j + 1][::-1] + ruta[j + 1:]


def vecino_aleatorio(ruta):
"""Elige al azar dos posiciones y devuelve el vecino 2-opt correspondiente.
Se usa en Simulated Annealing (que evalua un solo vecino por iteracion)."""
# Se empieza en 1 para mantener fija la primera ciudad de la ruta
i, j = sorted(random.sample(range(1, N), 2))
return vecino_2opt(ruta, i, j)


# ----------------------------------------------------------------------
# 3. HILL CLIMBING (descenso por la mejor mejora, al minimizar)
# ----------------------------------------------------------------------
def hill_climbing(ruta_ini):
"""Hill Climbing de mejor vecino. Devuelve (ruta, costo, historial)."""
actual = ruta_ini[:] # copia para no modificar la ruta original
c_act = costo(actual) # costo de la solucion actual
historial = [c_act] # guarda el costo en cada paso (para graficar)
mejora = True
while mejora: # se repite mientras exista un vecino mejor
mejora = False
mejor_v, mejor_c = None, c_act
# Se exploran TODOS los vecinos 2-opt de la solucion actual
for i in range(1, N - 1):
for j in range(i + 1, N):
v = vecino_2opt(actual, i, j)
c = costo(v)
# Se guarda el vecino que mas reduce el costo
# (1e-9 evita ciclos por errores de redondeo)
```

```
if c < mejor_c - 1e-9:
mejor_v, mejor_c = v, c
# Si hubo mejora, nos movemos al mejor vecino y seguimos
if mejor_v is not None:
actual, c_act = mejor_v, mejor_c
historial.append(c_act)
mejora = True
# Si ningun vecino mejora, se llego a un optimo local y el ciclo termina
return actual, c_act, historial


def hill_climbing_reinicios(n_reinicios, semilla):
"""HC con reinicios aleatorios para escapar de optimos locales.
Ejecuta HC varias veces desde rutas iniciales distintas y se queda con la mejor."""
random.seed(semilla) # semilla fija para que el experimento sea reproducible
mejor, mejor_c, mejor_h = None, float("inf"), []
hist_global = [] # mejor costo acumulado (para la grafica de convergencia)
mejor_global = float("inf")
for _ in range(n_reinicios):
ini = list(range(N))
random.shuffle(ini) # ruta inicial aleatoria
r, c, h = hill_climbing(ini)
for x in h:
mejor_global = min(mejor_global, x)
hist_global.append(mejor_global)
if c < mejor_c: # conservamos la mejor solucion de todos los reinicios
mejor, mejor_c = r, c
return mejor, mejor_c, hist_global


def hill_climbing_simple(semilla):
"""HC con un solo arranque aleatorio (puede quedar atrapado en optimo local)."""
random.seed(semilla)
ini = list(range(N))
random.shuffle(ini)
r, c, h = hill_climbing(ini)
return r, c, h


# ----------------------------------------------------------------------
# 4. SIMULATED ANNEALING (recocido simulado)
# ----------------------------------------------------------------------
def simulated_annealing(T0=2000.0, alpha=0.995, iteraciones=20000,
T_min=1e-3, semilla=0):
"""
T0: temperatura inicial, alpha: factor de enfriamiento geometrico (eta),
iteraciones: maximo de iteraciones, T_min: temperatura minima.
```

```
Criterio de Metropolis: acepta una solucion peor con prob exp(-delta/T).
"""
random.seed(semilla)
actual = list(range(N))
random.shuffle(actual) # solucion inicial aleatoria
c_act = costo(actual)
mejor, c_mejor = actual[:], c_act # mejor solucion vista hasta ahora
T = T0 # temperatura actual
hist_mejor, hist_actual = [c_mejor], [c_act] # historiales para graficar
for _ in range(iteraciones):
if T < T_min: # criterio de paro por temperatura minima
break
v = vecino_aleatorio(actual) # genera UN vecino al azar
c_v = costo(v)
delta = c_v - c_act # delta > 0 significa que el vecino es peor
# Si mejora (delta < 0) se acepta siempre. Si empeora, se acepta con
# probabilidad exp(-delta/T): alta con T grande, baja con T pequena.
if delta < 0 or random.random() < math.exp(-delta / T):
actual, c_act = v, c_v
if c_act < c_mejor: # actualiza el mejor global
mejor, c_mejor = actual[:], c_act
T *= alpha # enfriamiento geometrico: T = alpha * T
hist_mejor.append(c_mejor)
hist_actual.append(c_act)
return mejor, c_mejor, hist_mejor, hist_actual


# ---------------------------------------------------------------------------
# 5. EXPERIMENTOS Y GRAFICAS
# ---------------------------------------------------------------------------
def graficar_ruta(ax, ruta, titulo):
"""Dibuja la ruta sobre el plano longitud-latitud, regresando al inicio."""
xs = [CIUDADES[NOMBRES[i]][1] for i in ruta + [ruta[0]]] # longitudes
ys = [CIUDADES[NOMBRES[i]][0] for i in ruta + [ruta[0]]] # latitudes
ax.plot(xs, ys, "-o", lw=1.6, ms=5)
for i in ruta: # etiqueta con el nombre de cada ciudad
ax.annotate(NOMBRES[i], (CIUDADES[NOMBRES[i]][1],
CIUDADES[NOMBRES[i]][0]),
fontsize=6.5, xytext=(3, 3), textcoords="offset points")
ax.set_title(titulo, fontsize=10)
ax.set_xlabel("Longitud")
ax.set_ylabel("Latitud")
ax.grid(alpha=0.3)


def main():
# ---- Configuracion del experimento ----
```

```
SEMILLAS = list(range(30)) # 30 corridas independientes por algoritmo
N_REINICIOS = 5 # reinicios del Hill Climbing con reinicios
PARAM_SA = dict(T0=2000.0, alpha=0.9995, iteraciones=40000) # parametros de SA

# Contenedores de resultados y tiempos por algoritmo
# HC1 = Hill Climbing simple, HC = con reinicios, SA = Simulated Annealing
res = {"HC1": [], "HC": [], "SA": []}
tiempos = {"HC1": [], "HC": [], "SA": []}
best_hc1 = (None, float("inf"), None)
best_hc = (None, float("inf"), None)
best_sa = (None, float("inf"), None, None)

# ---- Ejecucion: cada semilla corre los 3 algoritmos ----
for s in SEMILLAS:
t = time.time()
r, c, h = hill_climbing_simple(s)
tiempos["HC1"].append(time.time() - t)
res["HC1"].append(c)
if c < best_hc1[1]: # guarda la mejor corrida (para graficas)
best_hc1 = (r, c, h)

t = time.time()
r, c, h = hill_climbing_reinicios(N_REINICIOS, s)
tiempos["HC"].append(time.time() - t)
res["HC"].append(c)
if c < best_hc[1]:
best_hc = (r, c, h)

t = time.time()
r, c, hm, ha = simulated_annealing(semilla=s, **PARAM_SA)
tiempos["SA"].append(time.time() - t)
res["SA"].append(c)
if c < best_sa[1]:
best_sa = (r, c, hm, ha)

# ---- Tabla de resultados: estadisticas por algoritmo ----
resumen = {}
for k in ("HC1", "HC", "SA"):
v = res[k]
resumen[k] = {
"mejor": min(v), "peor": max(v), "media": statistics.mean (v),
"desv": statistics.stdev (v), "tiempo_medio_s": statistics.mean (tiempos[k]),
"veces_mejor_global": None,
}
# Mejor costo encontrado por cualquiera de los metodos (referencia)
optimo_ref = min(min(res["HC1"]), min(res["HC"]), min(res["SA"]))
```

```
# Cuantas corridas de cada algoritmo igualaron esa referencia
for k in ("HC1", "HC", "SA"):
resumen[k]["veces_mejor_global"] = sum(1 for x in res[k] if abs(x - optimo_ref) <
1e-6)
resumen["referencia_mejor_global"] = optimo_ref
resumen["n_semillas"] = len (SEMILLAS)

# Ruta inicial aleatoria como punto de comparacion (semilla 0)
random.seed (0)
ini = list(range(N))
random.shuffle (ini)
resumen["costo_ruta_aleatoria"] = costo(ini)
resumen["ruta_HC1"] = [NOMBRES[i] for i in best_hc1[0]]
resumen["ruta_HC"] = [NOMBRES[i] for i in best_hc[0]]
resumen["ruta_SA"] = [NOMBRES[i] for i in best_sa[0]]
resumen["params_SA"] = PARAM_SA
resumen["reinicios_HC"] = N_REINICIOS

with open ("resultados.json", "w", encoding="utf-8") as f:
json.dump(resumen, f, indent=2, ensure_ascii=False)

# ---- Grafica 1: convergencia de HC (izq.) y SA (der.) ----
fig, axs = plt.subplots(1, 2, figsize=(11, 4))
axs[0].plot (best_hc1[2], color="tab:green", marker="o", ms=3, label="HC simple")
axs[0].plot (best_hc[2], color="tab:blue", label="HC con reinicios")
axs[0].legend()
axs[0].set_title("Convergencia Hill Climbing")
axs[0].set_xlabel("Paso de mejora acumulado")
axs[0].set_ylabel("Mejor costo (km)")
axs[0].grid(alpha=0.3)
# En SA se ve el costo actual (ruidoso: acepta peores) y el mejor costo
axs [1].plot (best_sa [3], color="tab:orange", alpha=0.4, lw=0.6, label="Costo
actual")
axs [1].plot (best_sa [2], color="tab:red", lw=1.6, label="Mejor costo")
axs[1].set_title("Convergencia Simulated Annealing")
axs[1].set_xlabel("Iteracion")
axs[1].set_ylabel("Costo (km)")
axs[1].legend()
axs[1].grid(alpha=0.3)
fig.tight_layout()
fig.savefig("convergencia.png", dpi=170)
plt.close(fig)

# ---- Grafica 2: comparacion de resultados finales (boxplot) ----
fig, ax = plt.subplots (figsize = (6.5, 4))
ax.boxplot ([res["HC1"], res["HC"], res["SA"]], tick_labels=["HC simple", "HC 5
reinicios", "Sim. Annealing"])
```

```
ax.set_ylabel("Costo final (km), 30 semillas")
ax.set_title("Comparacion de resultados")
ax.grid(alpha=0.3)
fig.tight_layout ()
fig.savefig("comparacion.png", dpi=170)
plt.close(fig)

# ---- Grafica 3: mejores rutas encontradas ----
fig, axs = plt.subplots (1, 2, figsize=(11, 4.6))
graficar_ruta (axs [0], best_hc [0], f"Mejor ruta HC (reinicios): {best_hc[1]:.1f} km")
graficar_ruta (axs [1], best_sa [0], f"Mejor ruta SA: {best_sa[1]:.1f} km")
fig.tight_layout ()
fig.savefig ("rutas.png", dpi=170)
plt.close(fig)

# ---- Analisis de sensibilidad: efecto de T0 y alpha en SA ----
configs = [(50, 0.99), (500, 0.99), (2000, 0.995), (2000, 0.999)]
sens = []
for T0, a in configs:
# 15 semillas por configuracion; [1] toma el costo final de cada corrida
vals = [simulated_annealing (T0=T0, alpha=a, iteraciones=40000, semilla=s) [1]
for s in range (15)]
sens.append({"T0": T0, "alpha": a, "media": statistics.mean(vals), "mejor":
min(vals)})
with open ("sensibilidad_sa.json", "w") as f:
json.dump (sens, f, indent=2)

# Imprime los resultados en consola
print (json.dumps (resumen, indent=2, ensure_ascii=False))
print (json.dumps (sens, indent=2))


# Punto de entrada: solo corre main() si se ejecuta el archivo directamente
if __name__ == "__main__":
main ()
```

Referencias

Jose Miguel. (2011, 28 de abril). ¿Cómo calcular la distancia entre dos puntos geográficos en C#? (Fórmula de Haversine). *Genbeta*. https://www.genbeta.com/desarrollo/como-calcular-la-distancia-entre-dos-puntos-geograficos-en-c-formula-de-haversine

Wikipedia. (s. f.-a). *Algoritmo de recocido simulado*. Recuperado el 21 de octubre de 2026, de https://es.wikipedia.org/wiki/Algoritmo_de_recocido_simulado

Wikipedia. (s. f.-b). *Algoritmo hill climbing*. Recuperado el 21 de octubre de 2026, de https://es.wikipedia.org/wiki/Algoritmo_hill_climbing

IA que utilice: Claude (Antropic, modelo Claude Sonnet 5.5

Prompt: Realiza la actividad Búsqueda local revisa lo que nos pide en el apartado producto y descripción, y que cumpla con las instrucciones generales y la lista de cotejo

Uso: Generación del código Python de los experimentos, graficas.

Verificación: El código se ejecuto con 30 semillas y los resultados del reporte provienen de esa ejecución (archivos resultados. json y sensibilidad_sa.json. Se ejecuto el código para comprobar los resultados

IA que utilice: Gemini (Google, 2026).

Prompt: Del código y de los datos que genera realiza qué puntos debe de ir en el documento.

Uso: Cumple el código con lo solicitado en la actividad 2.3 Búsqueda local y son correctos los datos, que datos se pueden poner en la documentación

Verificación: Para aclarar que la información sea correcta y para realizar la documentación correctamente